There are also a series of matchers based on expect { BLOCK 1 }.to change { BLOCK 2 }. The way this works is, RSpec runs block 2 and stores the value, then runs block 1, then checks block 2 again and the matcher passes if block 2's value has changed. You can chain additional methods to the end if you want to specify details, as in expect { book.publish! }.to change { book.publication_date }.from(nil).to(Date.today).

There's a lot more to RSpec matchers, including the ability to create your own. For more information, see the book *Effective Testing with RSpec 3* by Myron Marston and Ian Dees.

RSpec and Mocks

In addition to minitest, RSpec also allows you to create mock and stub objects, and has a lot of features available by default. Here's an overview of the most common.

In RSpec, the generic term for a fake object is *test double*, an object that stands in for another object, by analogy to "stunt double." The simplest way to create one is by using the method double. You can create the double and assign it a method to respond to using the RSpec method allow:

```
obj = double
allow(obj).to receive(:cost).and_return("cheap")
allow(obj).to receive(:name).and_return("banana")
obj.cost
obj.name
```

If you pass multiple arguments to and_return you specify responses for multiple times that the method is called. You can limit the arguments under which the double is invoked by chaining in .with, as in allow(obj).to receive(:availability).with("January").and_return(true).

You can define multiple responses at once with the receive_messages method, that takes a hash—keys become the methods to respond to, values are the fake values returned:

```
obj = double
allow(obj).to receive_messages(cost: "cheap", name: "banana")
obj.cost
obj.banana
```

Or you can use a shortcut by passing keyword arguments directly to double:

```
obj = double(cost: "cheap", name: "banana")
obj.cost
obj.banana
```

In minitest, we talked about *validating* mock objects by having the test fail if the method being faked is not called during the test. In RSpec, you manage this with the expect method.

You can call expect on a double before the main action of the spec, in this case expect is an exact replacement for allow:

```
obj = double
expect(obj).to receive(:cost).and_return("cheap")
expect(obj).to receive(:name).and_return("banana")
obj.cost
obj.banana
```